

A SUMMER INTERNSHIP PROJECT REPORT ON

OPEN-SOURCE EV DIAGNOSTIC TOOL

A Modular Python-Based UDS Diagnostic Framework over CAN for EVs

Submitted in partial fulfilment of the Summer Internship at

KPIT APEX LAB

May 2026 – July 2026

Mentored by

Mr. Chiranth Varadaraj

Submitted by

Nandini Tayade — 612407189

Minal Tawari — 612407188

The logo for KPIT APEX LAB. It features the text "KPIT" in a large, bold, sans-serif font. To its right is a vertical line, followed by the text "APEX" in a smaller, bold, sans-serif font, with "LAB" in an even smaller, bold, sans-serif font directly beneath it. A light gray vertical bar is positioned to the left of the "KPIT" text, and a light gray horizontal line is positioned below the "APEX LAB" text.

**KPIT | APEX
LAB**

Abstract

The rapid transition to electric mobility has introduced a significant challenge in automotive maintenance: the prevalence of closed-source, proprietary diagnostic systems. These "black box" ecosystems limit accessibility for independent technicians, researchers, and students, hindering the democratization of vehicle health data. This project presents the design and development of an Open-Source EV Diagnostic Tool, an affordable and transparent solution engineered to interface directly with vehicle communication networks. Utilizing industry-standard protocols—specifically Controller Area Network (CAN) and Unified Diagnostic Services (UDS)—the tool provides a modular, hardware-agnostic framework, fault code extraction, and deep-level system analysis.

The research methodology followed an iterative, applied engineering approach, transitioning from Software-in-the-Loop (SIL) simulation using a virtual ECU and CAN bus environment to Hardware-in-the-Loop (HIL) validation using a PCAN-USB interface. This process ensured that the diagnostic software was robust, timing-accurate, and safe for physical implementation. The results demonstrate that the tool successfully captures and decodes critical EV parameters, such as battery State of Charge (SOC) and thermal metrics, without the need for restrictive OEM-specific hardware. By providing a transparent, community-driven platform, this project bridges the gap between basic OBD-II scanning and advanced professional-grade diagnostics.

Contents

List of Abbreviations	vi
1 Introduction	1
1.1 Preamble	1
1.2 Problem Definition	1
1.3 Objective of the project	2
1.4 In-Scope	2
2 Literature Review	3
2.1 Overview of Automotive Diagnostic Evolution	3
2.2 Theoretical Framework: CAN Bus and UDS	3
2.3 Comparative Analysis: Proprietary vs. Open-Source Tools	4
2.4 Gap in Existing Research	4
3 Methodology and Approach	6
3.1 Overview of Our Approach	6
3.2 Phase 1: Research and Understanding the Basics	6
3.3 Phase 2: Deciding the Software Architecture and Rules	6
3.4 Phase 3: Writing the Code (Development)	8
3.5 Phase 4: Testing and Safety (Validation)	8
4 Implementation and Simulation	10
4.1 Repository Organisation	10
4.2 Configuration Layer	10
4.3 Virtual CAN Bus Simulation (Phase 1)	11
4.4 ECU Emulator	11
4.5 Diagnostic Application: CLI and GUI	12
4.6 Feature Implementation Status	13
5 Implementation and Simulation	16
5.1 Motivation for Hardware Migration	16
5.2 PCAN-USB Adapter Setup	16
5.3 Configuration-Only Migration	16
5.4 Session Keep-Alive Under Real Timing Constraints	17
5.5 Verification Using PCAN-View	18

6	Results and Discussions	19
6.1	Test Methodology	19
6.2	Demonstration Scenarios	19
6.3	Comparison with Proprietary Diagnostic Systems	20
6.4	Discussion	21
7	Summary and Conclusions	23
7.1	Summary of Work Done	23
7.2	Key Contributions	23
7.3	Limitations	23
7.4	Future Scope	24
7.5	Conclusion	24

List of Figures

3.1	: Layered software architecture of the Open-Source EV Diagnostic Tool.	7
3.2	: Layered software architecture of the Open-Source EV Diagnostic Tool.	9
3.3	: Layered software architecture of the Open-Source EV Diagnostic Tool.	9
4.1	Simulation-phase block diagram (virtual CAN bus loopback).	11
4.2	End-to-end data flow of a diagnostic request/response cycle.	12
4.3	Mock ECU Emulator GUI — bus logs and DTC/response-mode settings panel.	13
4.4	Diagnostic Tester GUI — raw UDS request entry, bus logs, and session status panel.	14
5.1	setting by the hardware CAN bus using PCAN-USB adapter	17
5.2	overall setup with the laptop	18

List of Tables

2.1	Comparison of Proprietary vs. Open-Source Diagnostic Systems	4
3.1	Core Layer/Module Responsibilities	7
4.1	Project Directory Structure	10
4.2	Feature Implementation Status	15
5.1	Configuration Parameters: Simulation vs. Hardware	16
6.1	Layer-by-Layer Verification and Test Cases	19
6.2	Proprietary vs. Open-Source EV Diagnostic Tool Comparison	21

List of Abbreviations

Abbreviation	Full Form
CAN	Controller Area Network
DBC	Database CAN
DID	Data Identifier
DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
EV	Electric Vehicle
GUI	Graphical User Interface
HIL	Hardware-in-the Loop
ISO	International Org. for Standardization
ISO-TP	ISO Transport Protocol
NRC	Negative Response Code
OBD	On-Board Diagnostics
OEM	Original Equipment Manufacturer
PCAN	Peak CAN
PCI	Protocol Control Information
SID	Service Identifier
SIL	Software-in-the Loop
SOC	State of Charge
UDS	Unified Diagnostic Services
	Services

CHAPTER 1

Introduction

1.1 Preamble

The modern automotive industry stands at a critical juncture, defined by the rapid convergence of mechanical engineering and advanced computer science. As the global transportation sector pivots toward widespread electrification, the character of the automobile itself has fundamentally changed. No longer purely mechanical, the modern Electric Vehicle (EV) is a sophisticated, data-driven ecosystem. These vehicles rely on intricate networks of sensors and embedded controllers that constantly generate high-fidelity telemetry, transforming the car into a mobile data hub. This transition into the era of the "Software-Defined Vehicle" has brought unparalleled efficiency and performance, yet it has simultaneously introduced an unintended consequence: the progressive isolation of the vehicle's internal intelligence from its owner and independent maintainers.

While the advancement of automotive software has accelerated the pace of innovation, it has also facilitated the rise of proprietary diagnostic architectures. These closed-source systems, governed by manufacturer-specific protocols, have transformed diagnostic data into a restricted asset. This digital divide between the Original Equipment Manufacturer (OEM) and the end-user has sparked global debate regarding the "Right to Repair" and the ethical implications of gatekeeping information essential for safety and maintenance. The dependency on centralized, manufacturer-mandated service tools is no longer merely a logistical challenge; it is a structural impediment to the long-term viability of the electric vehicle market.

To ensure the longevity and reliability of the growing EV fleet, there is an urgent imperative to re-evaluate how diagnostic data is accessed and interpreted. A move toward open-source diagnostic frameworks represents a vital correction to this trend. By fostering transparency, encouraging modular software design, and utilizing standardized international communication protocols, the industry can transition from an era of restricted maintenance to one of collaborative innovation. This report explores that transition, detailing the design of an open-source diagnostic platform that seeks to reclaim the transparency of automotive telemetry, ensuring that the vehicles of the future remain serviceable, adaptable, and truly transparent to those who operate and maintain them.

1.2 Problem Definition

Develop a minimal, cross-platform open-source diagnostic application that can:

- Communicate with an EV's CAN bus (11-bit/29-bit, configurable baud rate).

- Implement basic UDS services for reading ECU data and fault codes.
- Read and display generic EV diagnostic information such as voltage, current, SOC, temperatures, and motor parameters.
- Read, display, and clear generic diagnostic trouble codes (DTCs) from BMS/MC/VCU.
- Provide a simple dashboard UI for real-time data visualization and DTC management.
- Allow future expansion via plug-ins to support OEM-specific PIDs.

1.3 Objective of the project

- Implement a robust CAN/ISO-TP interface using SocketCAN or an open CAN-USB adapter (CANable/CANtact).
- Build a UDS-based diagnostic engine supporting key services: 0x10, 0x22, 0x19, 0x14, 0x3E.
- Create a lightweight desktop UI (Qt/Electron/Python-Qt) showing live data, DTC list, and clear-code functionality.
- Develop a small generic EV profile defining standard PIDs (voltage, SOC, temperature, motor RPM, etc.).
- Provide documentation: setup guide, API description, data model, and contribution workflow.

1.4 In-Scope

- CAN communication using open CAN adapters.
- UDS over ISO-TP (core services only).
- Reading/clearing DTCs (generic implementation).
- Live data visualization (chart + numeric view).
- Session logging and optional export to JSON/CSV.
- Simple plug-in mechanism for future OEM extensions.

CHAPTER 2

Literature Review

2.1 Overview of Automotive Diagnostic Evolution

The rapid evolution of Electric Vehicles (EVs) has necessitated a shift from mechanical-centric troubleshooting to software-defined diagnostics. Traditional Internal Combustion Engine (ICE) vehicles relied on mechanical inspections and simple OBD-II fault codes. In contrast, EVs are complex, software-heavy systems where battery health, thermal management, and power electronics are managed by interconnected Electronic Control Units (ECUs). Literature indicates that proprietary diagnostic systems, while reliable, create "vendor lock-in," limiting the ability of independent technicians and researchers to access critical vehicle data. This project explores the transition toward open-source diagnostic architectures as a means to democratize access to this information.

2.2 Theoretical Framework: CAN Bus and UDS

Theoretical Framework: Automotive Communication Standards The diagnostic architecture of modern Electric Vehicles (EVs) relies on a layered communication model. This project is built upon three foundational ISO standards that ensure interoperability between diagnostic hardware and vehicle Electronic Control Units (ECUs).

Controller Area Network (CAN) – ISO 11898: The CAN bus serves as the vehicle's nervous system. It is a multi-master, message-based serial communication protocol. Its defining characteristic is its ability to handle "arbitration"—when multiple ECUs attempt to broadcast simultaneously, the CAN protocol uses an identifier-based priority system to ensure that critical messages (like battery safety alerts) are delivered without latency. For our diagnostic tool, CAN acts as the physical and data link layer, allowing us to "listen" to the high-speed bus traffic (typically 500 kbps to 1 Mbps) found in EV networks without interrupting the vehicle's operation.

Unified Diagnostic Services (UDS) – ISO 14229: If CAN is the transmission medium, UDS is the semantic language. Standardized under ISO 14229, UDS operates at the application layer, facilitating a rigid "Client-Server" relationship. In this model, our diagnostic tool acts as the client and the vehicle's ECU as the server. The power of UDS lies in its versatility: it manages session control, security access, and specific service identifiers (SIDs). By utilizing SIDs such as Read Data By Identifier (0x22), our tool can query specific DIDs that map directly to high-value battery metrics like voltage, state-of-charge (SOC), and cell-level temperature.

Transport Layer Protocol (ISO-TP) – ISO 15765-2: Standard CAN frames are limited to an

8-byte payload. Modern diagnostic messages, however, often require much larger data structures. ISO 15765-2 (ISO-TP) solves this limitation by acting as the transport layer. It provides the logic for "segmentation"—splitting a large UDS message into smaller chunks that fit into 8-byte CAN frames—and "reassembly"—recombining these chunks at the destination. Our implementation relies on ISO-TP to ensure that complex diagnostic requests are transmitted successfully without data corruption.

2.3 Comparative Analysis: Proprietary vs. Open-Source Tools

Academic and industry literature highlight a clear distinction between existing diagnostic paradigms:

Feature	Proprietary Diagnostic Systems	Open-Source Diagnostic Initiatives
Accessibility	Restricted; usually requires dealership authorization.	Universal; fostered by community collaboration.
Cost	Extremely high; annual licensing and hardware fees.	Low; based on accessible hardware/libraries.
Transparency	Closed "Black Box"; protocols are proprietary.	Transparent; code is peer-reviewed and auditable.
Customization	Rigid; software is locked to OEM platforms.	Modular; extensible for custom metrics.
Vendor Independence	High dependency; functionality ends if support ceases.	High autonomy; community-maintained.

Table 2.1: Comparison of Proprietary vs. Open-Source Diagnostic Systems

Current literature suggests that while proprietary tools offer polished interfaces and manufacturer-backed support, they stifle innovation. Open-source initiatives, by contrast, offer a platform for "Right to Repair" initiatives, security auditing through transparency, and the ability to tailor diagnostic dashboards for specific research or maintenance needs.

2.4 Gap in Existing Research

While existing research confirms that OBD-II scanners can perform basic tasks, they lack the "deep-level" diagnostic capabilities—such as cell-level battery monitoring and advanced thermal management diagnostics—required for modern EVs. Most academic studies on UDS implementation focus on laboratory-based software development, with a notable gap in low-cost, hardware-validated open-source tools that can seamlessly bridge the gap between simulation and real-world vehicle testing. This project addresses this gap by demonstrating a validated workflow from virtual ECU simulation to physical implementation using standardized inter-

faces like PCAN-USB.

CHAPTER 3

Methodology and Approach

3.1 Overview of Our Approach

Building an open-source Electric Vehicle (EV) diagnostic tool requires a clear, step-by-step plan. Our goal was to create a tool that can connect to a car's internal computer network, ask for health data, and display it clearly. To make sure the project was organized and successful, we divided our work into four main phases: researching existing tools, choosing the right hardware, writing the software, and testing the final product. This step-by-step approach ensured that our tool would be safe, accurate, and easy for other developers to use and improve.

3.2 Phase 1: Research and Understanding the Basics

Before building anything, we looked at how car mechanics currently diagnose electric vehicles. We found that most car manufacturers use private, locked-down tools. These tools are very expensive and do not let independent mechanics or students see how they work. We decided our tool needed to be the exact opposite: free, open, and transparent. To understand how to talk to an EV, we had to study the standard languages cars use. We focused heavily on two main systems:

- **CAN Bus (Controller Area Network):** This is the physical network of wires inside the car that lets all the different computers (like the battery manager and the motor controller) talk to each other.
- **UDS (Unified Diagnostic Services):** This is the standard rulebook for asking the car's computers for information. It tells us exactly how to ask the car questions like, "Are there any broken sensors?" or "What is the current battery level?"

3.3 Phase 2: Deciding the Software Architecture and Rules

Once we understood how cars communicate, we had to determine the software architecture and the foundational rules for our tool.

- **Software Architecture:** The resulting architecture is organised into six layers, implemented as independent modules inside a shared core package, as shown in Figure 3.1.

The responsibility of each layer is summarised in the table 3.1 :

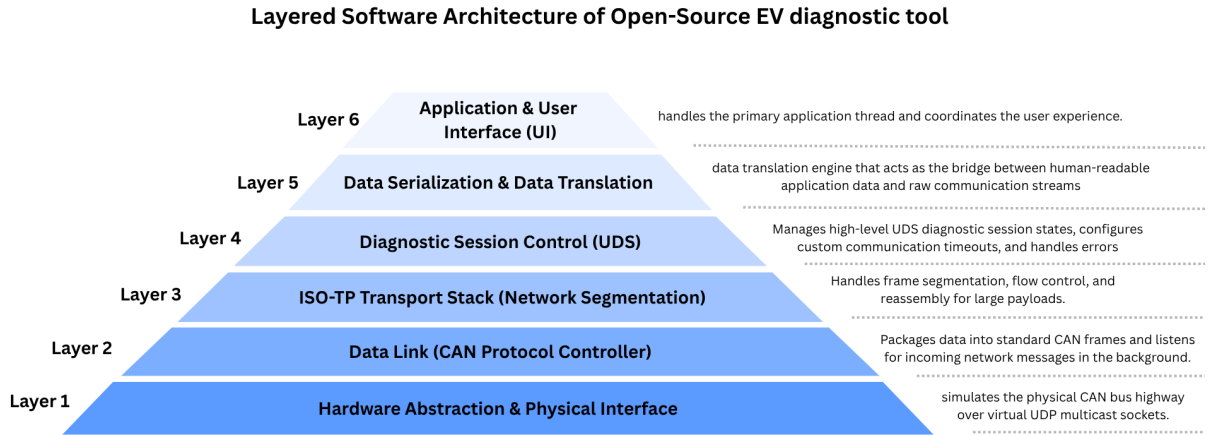


Figure 3.1: : Layered software architecture of the Open-Source EV Diagnostic Tool.

Layer / Module	Responsibility
<code>can_interface.py</code>	Opens and configures the physical or virtual CAN bus (python-can Bus object) using interface, channel and bitrate parameters.
<code>transport_layer.py</code>	Builds ISO-TP physical and functional CanStack objects (can-isotp) for segmentation/reassembly of multi-frame diagnostic messages.
<code>uds_layer.py</code>	Implements UDS response evaluation: matches SID+0x40 positive responses, detects 0x7F negative responses, and decodes NRC bytes via a standard lookup dictionary.
<code>diagnostic_engine.py</code>	Orchestrates <code>send()/receive()</code> over the ISO-TP stack with timeout handling; the single point through which all diagnostic requests are issued.
<code>translator.py</code>	Maps raw response payloads to human-readable engineering values using scaling rules (factor, offset, unit) stored in <code>decode_values.xlsx</code> .
<code>tester_present.py</code>	Runs a background daemon thread that transmits the Tester-Present (0x3E 0x80) frame every 2 seconds to keep an Extended/Programming diagnostic session alive.

Table 3.1: Core Layer/Module Responsibilities

- **Deciding the Software:** We prioritized a modular software design to ensure the tool remains flexible and easy for the open-source community to update. We chose Python for

its efficiency in data processing and its extensive library support for automotive communication. The software is structured into two primary layers:

- o **Communication Layer:** This handles the timing, reception, and transmission of CAN messages, ensuring we interact with the vehicle bus safely and accurately.
- o **Data Interpretation Layer:** This processes the raw hex codes and converts them into meaningful diagnostic values, such as battery voltage, State of Charge (SOC), and temperature.
- **Setting the Rules:** We programmed our tool to use specific Unified Diagnostic Services (UDS) commands. We focused on the commands most critical for EV maintenance. For example, we implemented specific request codes to pull real-time data and diagnostic commands to read and clear error messages (DTCs) if a system component fails.

3.4 Phase 3: Writing the Code (Development)

With the software architecture and UDS rules defined, we moved on to writing the actual code. We built the software in small, manageable pieces to ensure reliability and ease of maintenance.

- **Translating the Data:** When a car sends data over the CAN bus, it appears as a long, cryptic string of hexadecimal numbers. We wrote software to capture these raw strings and translate them into plain English. For example, the software automatically parses the incoming raw data and displays it on the screen in a human-readable format.
- **Open-Source Sharing:** Because this is an open-source project, our documentation process was as important as the code itself. We did not simply keep the code on our own computers; we organized it neatly and wrote clear, comprehensive instructions explaining how the logic works. This ensures that other students, engineers, or hobbyists can easily download our project, understand the underlying mechanisms, and confidently add their own features or improvements

3.5 Phase 4: Testing and Safety (Validation)

Testing was the most critical part of the project to ensure the tool could communicate reliably without interfering with the vehicle's own systems. We followed a transition from virtual testing to hardware testing:

- **Virtual Simulation:** We began by simulating a "Virtual ECU" and a "Virtual CAN" bus on our development computer. This allowed us to build and refine the logic of our code in a safe, software-only environment. We created virtual signals that mimicked an actual EV's behavior to ensure our diagnostic tool could correctly "listen" and interpret the simulated data without needing any physical hardware, shown in fig.3.2.

Figure 3.2: : Layered software architecture of the Open-Source EV Diagnostic Tool.

- **Transition to Hardware:** Once the code was stable in the virtual environment, we replaced the "Virtual CAN" with a PCAN-USB interface. This allowed us to bridge our computer-based software with actual hardware components. This was the crucial "bridge" test—it proved that the software we wrote was not just working in a simulator, but was fully capable of interacting with real-world CAN signals through hardware shown in fig.3.3.

Figure 3.3: : Layered software architecture of the Open-Source EV Diagnostic Tool.

- **Final Verification:** With the PCAN-USB confirming that our code could run on hardware, we were able to verify the timing and data-handling capabilities of the tool, ensuring it could handle the real-world communication speeds required for professional EV diagnostics.

CHAPTER 4

Implementation and Simulation

4.1 Repository Organisation

The codebase is organised into six functional workspaces so that documentation, configuration, protocol logic, simulation, applications and testing remain cleanly separated:

Directory	Purpose
Architecture_and_Docs/	Block diagrams, layered-architecture diagrams, code-flow PDF, and feature/test-scenario trackers.
config/	Global configuration constants (<code>user_config.py</code>), dynamic path handling (<code>file_paths.py</code>), and Excel lookup tables (<code>ev_state_final.xlsx</code> , <code>decode_values.xlsx</code>).
core/	The protocol stack: <code>can_interface</code> , <code>transport_layer</code> , <code>uds_layer</code> , <code>diagnostic_engine</code> , <code>translator</code> , <code>tester_present</code> .
ECU_Simulations/	The virtual ECU node (<code>ecu.py</code>) and its GUI (<code>gui_ecu.py</code>) — an offline sandbox that removes the need for a physical vehicle during development.
Diagnostic_Applications/	The tester application: CLI (<code>main.py</code>) and PyQt5 GUI (<code>gui_main.py</code>).
Testing_and_Validation/	Layer-by-layer verification scripts (<code>layer1</code> – <code>layer5</code>) validating each part of the stack in isolation before integration.

Table 4.1: Project Directory Structure

4.2 Configuration Layer

`userconfig.py` defines every parameter that changes between deployments: physical addressing (`TARGETTXID = 0x7E0`, `TARGETRXID = 0x7E8`), the broadcast functional address (`FUNCTIONALID = 0x7DF`), the CAN interface type and channel, the bus bitrate, ISO-TP timing parameters (`STmin`, block size, padding byte), the list of supported service identifiers, and diagnostic timeouts. Keeping these values in one file — rather than scattered through application code — is what allows the same core modules to run against either the simulated or the physical bus. Two Excel workbooks act as the tool’s data dictionary: `evstatefinal.xlsx` stores the ECU-side mock state (the raw hex payload the simulated ECU should return for a

given SID/DID combination), and `decodevalues.xlsx` stores the tester-side decoding rules — the DID, a scaling factor, an offset and a unit string — used by `translator.py` to turn a raw payload into a labelled physical value (for example "HVVoltage: 380 V").

4.3 Virtual CAN Bus Simulation (Phase 1)

Rather than depending on `SocketCAN` — which is only available on Linux and would have restricted where the tool could be developed and demonstrated — Phase 1 used `python-can`'s UDP-multicast virtual bus interface. This lets the tester application and the ECU simulator exchange genuine CAN frames over a loopback network interface, fully exercising the ISO-TP segmentation and UDS request/response logic, while remaining completely operating-system independent and requiring no vehicle hardware. Figure 4.1 illustrates the simulation block diagram, and Figure 4.2 traces a single diagnostic request end-to-end through all six layers.

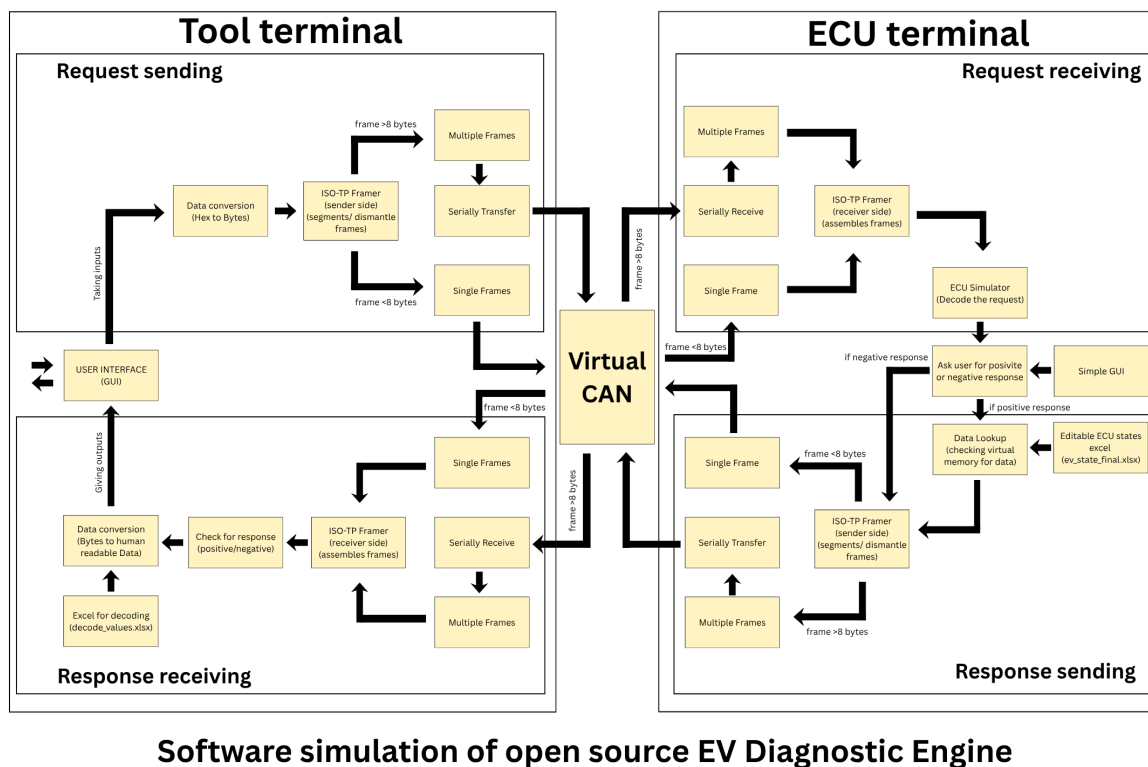


Figure 4.1: Simulation-phase block diagram (virtual CAN bus loopback).

4.4 ECU Emulator

The ECU emulator (`ECUSimulations/ecu.py`) models a virtual EV control unit that listens on its physical (`0x7E0`) and functional (`0x7DF`) ISO-TP addresses. It validates incoming service identifiers against the supported set (`0x10`, `0x22`, `0x3E`, `0x19`, `0x14`), responds to Tester Present frames without printing session-cluttering output, and — for Read Data By Identifier requests

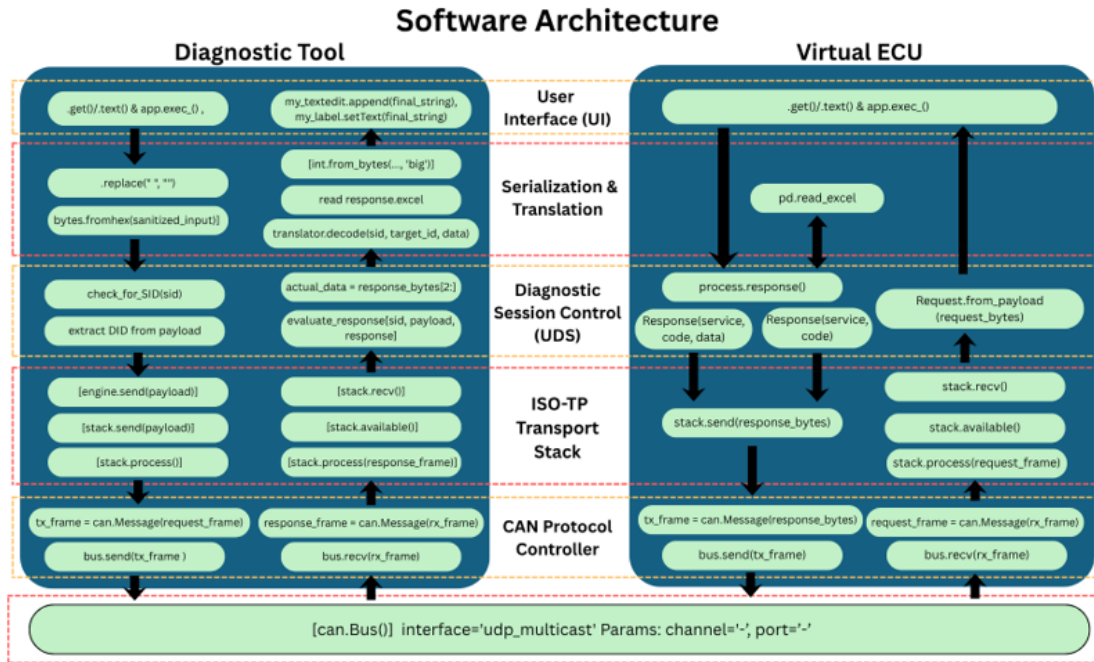


Figure 4.2: End-to-end data flow of a diagnostic request/response cycle.

— looks up the appropriate raw payload from `evstatefinal.xlsx` and returns it as a positive response. Two features give the emulator realistic fault-injection behaviour that a technician would need when validating a tester application:

- A runtime response-mode switch (keys 'p' / 'n') that forces either genuine positive responses or randomised Negative Response Codes drawn from a realistic NRC pool (0x10, 0x11, 0x12, 0x13, 0x22, 0x31, 0x33, 0x7E).
- A selectable Diagnostic Trouble Code menu exposed on Service 0x19 requests, offering five representative EV fault codes — P0A80 (Replace Hybrid/EV Battery Pack), P0AA6 (Hybrid Battery Voltage System Isolation Fault), P0A1F (Battery Energy Control Module Performance), P0C2F (Internal Control Module Drive Motor Control Performance) and U0100 (Lost Communication With ECM/PCM) — any combination of which can be returned to the tester for a given test run.

A companion GUI (`guiecu.py`) exposes the same functionality — start/stop of the emulator, a live bus-log panel, and checkboxes for selecting DTCs to report — through a two-panel PyQt5 window (controls/logs on the left, response settings on the right), removing the need to interact with the emulator through the terminal, this is shown in fig 4.3.

4.5 Diagnostic Application: CLI and GUI

The tester side of the tool is provided as two independent, purpose-built entry points that share the same underlying core logic rather than being merged into a single interface:

- `main.py` — a command-line tester that initialises the hardware/virtual bus, builds the physical and functional ISO-TP stacks, accepts raw hex UDS commands typed by the

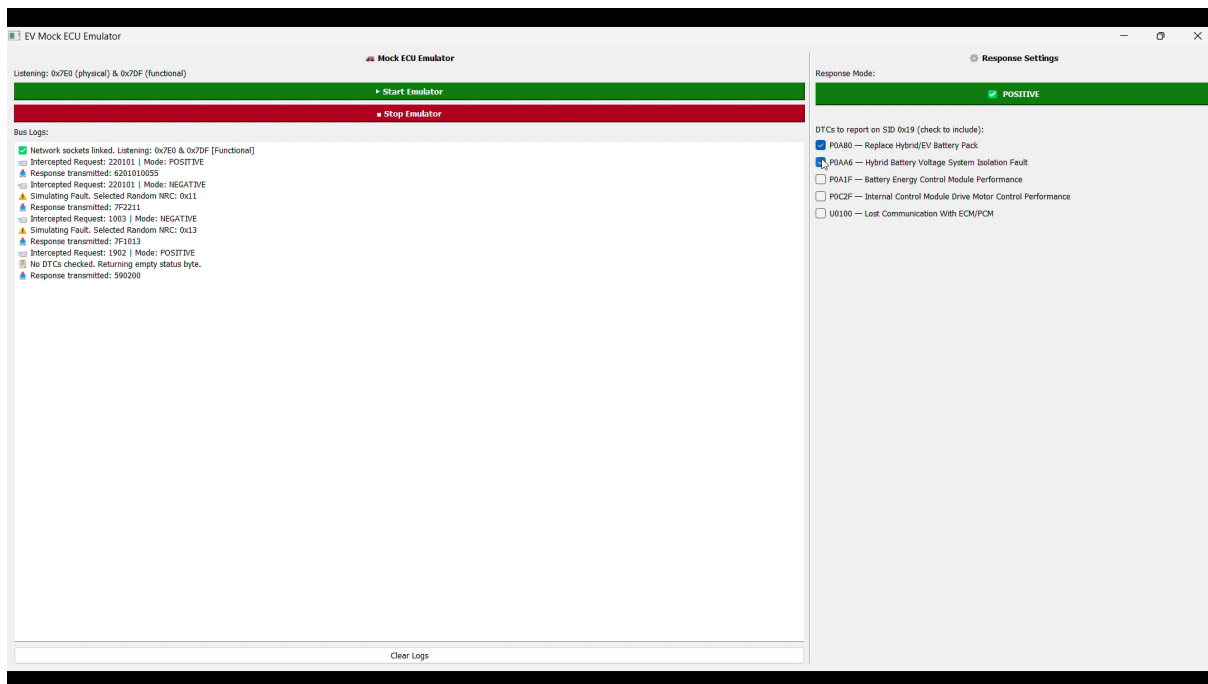


Figure 4.3: Mock ECU Emulator GUI — bus logs and DTC/response-mode settings panel.

operator (e.g. "22 02 01"), validates the SID against the supported list, transmits the request, waits for a response within `DEFAULTTIMEOUT`, prints the raw hex response wrapped at eight bytes per line, and — for a positive response — prints the translated human-readable value. It also manages session state: entering an Extended (0x03) or Programming (0x02) session automatically starts the `TesterPresentManager` background thread, and returning to the Default session (0x01) stops it.

- `guimain.py` — a PyQt5 graphical tester that mirrors the same send → receive → evaluate → decode pipeline used by the CLI, but executes it inside a `QThread`-based `UDSWorker` so that the diagnostic exchange never blocks the Qt event loop. Results are delivered back to the main window through a `pyqtSignal` and rendered in a two-panel layout: controls, raw request entry and bus logs on the left; TX/RX address configuration ("Apply Reconnect"), and session status on the right. And is shown in fig.4.4.

Both applications import identical core modules (`caninterface`, `transportlayer`, `diagnosticengine`, `udslayer`, `translator`, `testerpresent`), so a fix or feature added to the protocol stack is automatically available to both the CLI and the GUI without duplicated logic — a deliberate design choice to keep the CLI and GUI as separate, independent tools rather than merging them into one monolithic script.

4.6 Feature Implementation Status

Implementation progress was tracked feature-by-feature against both the tool and ECU sides using a shared status spreadsheet, an excerpt of which is reproduced in Table 4.1. It shows that all core protocol mechanics (bus initialisation, ISO-TP transmission/reception, physical and

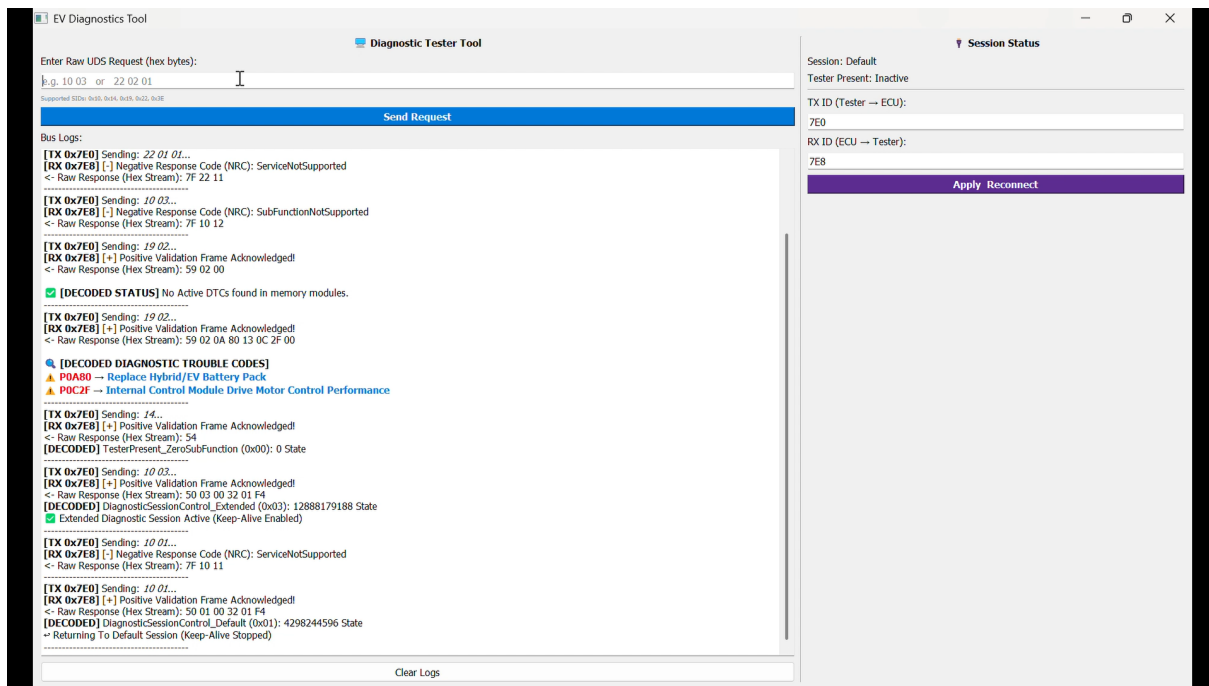


Figure 4.4: Diagnostic Tester GUI — raw UDS request entry, bus logs, and session status panel. functional addressing, positive/negative response validation, NRC decoding) and the primary Read Data By Identifier workflow — including all EV-specific parameters (HV voltage, SOC, SOH, pack temperature, VIN, ECU software version, pack current) — reached a Closed (fully implemented) status, while a small number of advanced features (multi-ECU support, persistent request/response history export) were scoped as future extensions.

#	Feature	Tool Status	ECU Status
1	CAN Bus Initialisation	Implemented	Implemented
2	ISO-TP Stack Creation (Physical & Functional)	Implemented	Implemented
3	Physical Addressing (0x7E0 ↔ 0x7E8)	Implemented	Implemented
4	Functional Addressing (0x7DF)	Implemented	Implemented
5	Positive / Negative Response Validation	Implemented	Implemented
6	NRC Decoding / Generation	Implemented	Implemented
7	SID 0x22 ReadDataByIdentifier (all EV DIDs)	Implemented	Implemented
8	SID 0x19 ReadDTCInformation	Implemented	Implemented
9	SID 0x3E TesterPresent keep-alive	Implemented	Implemented
10	SID 0x10 Diagnostic Session Control	Implemented	Implemented
11	SID 0x14 ClearDiagnosticInformation	Implemented	Implemented
12	GUI-based Tester and Emulator	Implemented	Implemented
13	Multi-ECU Support	Implemented	Implemented
14	Persistent Request/Response History	Partial	Not Implemented

Table 4.2: Feature Implementation Status

CHAPTER 5

Implementation and Simulation

5.1 Motivation for Hardware Migration

Simulation validates protocol logic but cannot confirm that the same software behaves correctly on a real electrical CAN network, where bus timing, termination, bit-rate negotiation and driver-level behaviour introduce constraints that a loopback simulation cannot reproduce. The second phase of the project therefore migrated the validated Phase-1 software onto a physical CAN bus using a PEAK-System PCAN-USB adapter, to demonstrate that the tool is genuinely usable for real EV diagnostics and not merely a simulation exercise.

5.2 PCAN-USB Adapter Setup

The PCAN–USB adapter was connected to the host PC over USB and to the vehicle/ECU network over its CAN-H/CAN-L pins, using PEAK-System’s PCAN–Basic driver stack. python-can natively supports this adapter through its ‘pcan’ interface backend, so no custom driver code was required — the existing `can.Bus()` call in `caninterface.py` simply needed to be pointed at the PCAN backend and the correct USB channel as shown in fig. 5.1 and fig 5.2.

5.3 Configuration-Only Migration

Because the architecture strictly confines all bus-specific detail to `config/userconfig.py`, moving from the simulated bus to the physical PCAN adapter required changing only two values:

Parameter	Simulation (Phase 1)	Hardware (Phase 2)
CAN_INTERFACE	udp_multicast / virtual	pcan
CAN_CHANNEL	loopback multicast group	PCAN_USBBUS1 (tool) / PCAN_USBBUS2 (ECU)
PORT/BITRATE	port-5001	bitrate-500000
ISO-TP parameters (<code>stmin</code> , <code>blocksize</code> , <code>padding</code>)	unchanged	unchanged
Physical / functional address- ing (<code>0x7E0/0x7E8/0x7DF</code>)	unchanged	unchanged

Table 5.1: Configuration Parameters: Simulation vs. Hardware



Figure 5.1: setting by the hardware CAN bus using PCAN-USB adapter

No changes were required in `transportlayer.py`, `udslayer.py`, `diagnosticengine.py`, `translator.py` or `testerpresent.py`, nor in either the CLI or GUI application — confirming that the layered design objective set out in Chapter 3 (“only bustype/channel should change between simulation and hardware”) was met in practice.

5.4 Session Keep-Alive Under Real Timing Constraints

On a physical bus, background `TesterPresent` traffic and operator-triggered diagnostic requests can genuinely occur close together in time, which surfaced a threading consideration that a slower, purely simulated environment did not stress in the same way: the `TesterPresentManager`’s keep-alive worker and a manually-issued request can both attempt to use the diagnostic engine concurrently. This was addressed by serialising access to the shared send/receive pathway (a single lock-guarded request cycle per ECU exchange) so that a keep-alive heartbeat and an operator-initiated request cannot interleave and corrupt each other’s ISO-TP transaction — an important correctness requirement once the tool is driven against real, timing-sensitive ECU hardware rather than a fully deterministic loopback simulation.



Figure 5.2: overall setup with the laptop

5.5 Verification Using PCAN-View

To independently confirm that the tool was generating protocol-correct traffic on the physical bus (rather than relying solely on the tool’s own logs), PEAK-System’s PCAN-View utility was run alongside the diagnostic tool. PCAN-View displays every raw CAN frame placed on the bus with its arbitration ID, DLC and data bytes, allowing the ISO-TP First-Frame/Consecutive-Frame segmentation, the UDS request bytes, and the ECU’s positive/negative response frames to be visually cross-checked against what the tool itself reported — providing an independent, tool-external verification of correct on-bus behaviour.

CHAPTER 6

Results and Discussions

6.1 Test Methodology

Verification followed a bottom-up, layer-by-layer strategy matching the six-layer architecture: each layer was tested in isolation with a dedicated script before the full application was exercised end-to-end. This mirrors standard practice for protocol-stack software, where defects are far cheaper to localise at the layer that introduces them than after full integration.

Layer	Verification Focus	Representative Test
Layer 1 (CAN)	Bus initialisation and raw frame transmit/receive	Tool and ECU bus open without exception; a broadcast frame (ID 0x7DF, data 11 22 33) sent by the tool is correctly captured by the ECU listener.
Layer 1–3 (ISO-TP)	Stack initialisation, single-frame and multi-frame (>8 byte) segmentation/reassembly	A 9-byte request (22 F1 90 AA BB CC DD EE FF) is automatically segmented by the tool stack and correctly reassembled by the ECU stack.
Layer 4 (UDS)	Positive/negative response validation for SIDs 0x10, 0x22, 0x14, 0x19, 0x3E; NRC decoding; timeout detection	Response 62 F1 90 41 42 43 to request 22 F1 90 is correctly matched as positive and stripped to payload 41 42 43; response 7F 22 22 is correctly decoded as ConditionsNot-Correct.
Layer 5 (Translator)	Hex sanitisation, ASCII decoding, scaled physical-value calculation, and graceful fallback when a DID or the Excel file is missing	Payload 574241 against DID 0xF190 is decoded as an ASCII VIN fragment; an unmapped DID falls back to a labelled raw-hex display instead of failing.

Table 6.1: Layer-by-Layer Verification and Test Cases

6.2 Demonstration Scenarios

Beyond the automated layer tests, the tool was demonstrated end-to-end under three representative scenarios:

- Single-ECU simulation: the tester GUI and the ECU emulator GUI are run together over

the virtual bus; a Read Data By Identifier request for HV pack voltage (DID 0x00A0) returns and correctly decodes a live value, and switching the emulator to negative-response mode demonstrates correct NRC handling on the tester side.

- Fault-injection / DTC scenario: the emulator's DTC menu is used to return one or more simulated fault codes (for example P0A80 — Replace Hybrid/EV Battery Pack) in response to a Service 0x19 request, and the tester correctly surfaces the returned DTC bytes.
- Hardware scenario: the identical tester and ECU applications are re-pointed at the PCAN-USB adapter (Chapter 5), and the same Read Data By Identifier and DTC workflows are repeated on the physical bus, with PCAN-View used to independently confirm the on-wire frames.

6.3 Comparison with Proprietary Diagnostic Systems

Table 6.2 summarises how the open-source tool compares against the general characteristics of commercial, proprietary EV/OEM diagnostic suites.

Aspect	Proprietary Diagnostic Systems	Open-Source EV Diagnostic Tool
Cost / Licensing	Paid licence, often per-seat or per-vehicle-brand	Free and open-source (MIT/community-style licensing)
Source code access	Closed-source; internals not inspectable	Fully open Python source, inspectable and modifiable
Hardware dependency	Often requires a specific vendor J2534/proprietary dongle	Works with any python-can-supported adapter (e.g. PCAN-USB); simulation mode needs no hardware at all
Extensibility	Fixed parameter set defined by vendor	New DIDs/parameters addable via an editable Excel sheet, no core code changes
Protocol coverage	Full UDS + OEM-specific extensions, security access, flashing	Core UDS services (0x10, 0x22, 0x19, 0x14, 0x3E); security access and reprogramming out of current scope
Offline development / training	Typically requires connection to a real or simulated vendor ECU	Built-in configurable ECU emulator enables full offline development and training
Transparency of diagnosis logic	Black-box scaling/decoding rules	Scaling factors, offsets and units are visible and editable in a spreadsheet

Table 6.2: Proprietary vs. Open-Source EV Diagnostic Tool Comparison

6.4 Discussion

The results confirm that a layered, protocol-faithful implementation of CAN/ISO-TP/UDS in Python is sufficient to reproduce the essential read/diagnose workflow of commercial EV diagnostic tools, at a fraction of the cost and with full source-level transparency. The clean separation between the core protocol stack and the two front-ends (CLI, GUI) proved valuable in practice: features and bug fixes made once in core/ were automatically available to both applications. The Excel-driven configuration approach met the mentor’s explicit requirement of avoiding hard-coded diagnostic parameters and made it straightforward to add new EV-specific DIDs without editing Python code.

The layer-by-layer test strategy proved effective at isolating faults early — for example, multi-frame segmentation issues were caught and fixed at Layer 3 before they could manifest as confusing symptoms at the application layer. The most significant lesson from the hardware phase was that a purely simulated bus, while excellent for functional development, does not fully exercise timing-sensitive interactions between background keep-alive traffic and operator-

issued requests; this was only surfaced — and resolved — once the tool was exercised on the physical PCAN bus, reinforcing the value of the project’s two-phase (simulate-then-validate-on-hardware) methodology.

Current limitations include the absence of Security Access (Service 0x27) and routine/ECU-reprogramming support, single-ECU-at-a-time operation in the present build, and console-only (non-persistent) logging in the ECU emulator. These are discussed further as future work in Chapter 7.

CHAPTER 7

Summary and Conclusions

7.1 Summary of Work Done

This project designed, implemented and validated a modular, open-source EV diagnostic tool that reproduces the core communication stack used by commercial UDS-based diagnostic systems: CAN bus access, ISO-TP transport segmentation, UDS request/response handling, tester-present session keep-alive, and human-readable translation of diagnostic payloads. The tool was built and demonstrated first in a fully virtual, hardware-independent simulation environment, complete with a configurable ECU emulator supporting both positive and fault-injected (negative-response and DTC) behaviour, and both command-line and PyQt5 graphical interfaces. It was then successfully migrated to run against real CAN hardware via a PCAN-USB adapter with configuration-only changes, and the resulting on-bus traffic was independently verified using PCAN-View. Verification was carried out using a structured, layer-by-layer test plan covering all six architectural layers.

7.2 Key Contributions

- A reusable, layered reference implementation of the CAN → ISO-TP → UDS diagnostic stack in Python, cleanly separated into independent, testable modules.
- A configurable virtual ECU emulator (CLI and GUI) enabling realistic, hardware-free training and development, including selectable DTC fault injection.
- Demonstrated, minimal-change portability between a simulated bus and a real PCAN-USB hardware bus, validating the project's core architectural objective.
- A spreadsheet-driven DID scaling/decoding mechanism that removes the need to hard-code vehicle-specific diagnostic parameters in application code.
- A structured, repeatable layer-by-layer test suite that can be reused for regression testing as the tool is extended

7.3 Limitations

- Security Access (Service 0x27) and ECU reprogramming/flashing (Routine Control, Request Download/Transfer Data) are not yet implemented, limiting the tool to read-oriented diagnostics.

- The current build supports one ECU node at a time; genuine multi-ECU bus arbitration and addressing is planned but not yet complete.
- Diagnostic activity logging is presently console-based and not persisted to a file or database for later audit/history review.

7.4 Future Scope

- Implement Security Access (0x27) and Routine Control (0x31) services to support a wider range of real EV diagnostic and calibration workflows.
- Extend the ECU emulator and tester application to support multiple simultaneous virtual/real ECUs on the same bus, closer to a real vehicle topology.
- Add persistent, exportable logging of diagnostic sessions (CSV/JSON) to support audit trails and offline analysis.
- Validate the tool against additional CAN adapters supported by python-can (e.g. Vector, Kvaser) to confirm hardware portability beyond the PCAN-USB adapter used in this project.
- Package the tool for easier distribution (e.g. as an installable Python package or standalone executable) to lower the barrier to adoption for hobbyists and independent workshops.

7.5 Conclusion

This project demonstrates that the essential diagnostic capability of expensive, closed-source EV diagnostic suites — reading live vehicle parameters, retrieving stored fault codes, and maintaining a diagnostic session — can be reproduced using an entirely open-source, well-architected Python application built on standard, published protocols (CAN, ISO-TP, UDS). By validating the design first in simulation and then on real hardware, the project shows that a hardware-agnostic, layered architecture is not just a theoretical best practice but a practical asset that measurably reduced the effort required to move from a virtual bus to a physical vehicle network. The resulting Open-Source EV Diagnostic Tool provides a solid, extensible foundation for further work in accessible, transparent electric vehicle diagnostics.

References

1. **ISO 11898-1:2015**

— Controller area network (CAN) — Part 1: Data link layer and physical signalling. International O

2. **ISO 15765-2:2016**

—Diagnostic communication over Controller Area Network (DoCAN) — Part 2: Transport protocol

3. **] ISO 14229-1:2020**

— Unified diagnostic services (UDS) — Part 1: Application layer. International Organization for Sta

4. **python-can documentation**

— Python Controller Area Network interface library. <https://python-can.readthedocs.io>

5. **can-isotp documentation**

— ISO-TP transport layer implementation for Python. <https://can-isotp.readthedocs.io>

6. **PEAK-System Technik GmbH**

— PCAN-USB Adapter and PCAN-View User Manual.

7. **openpyxl documentation**

— Python library for reading/writing Excel 2010 xlsx/xlsm files.